

CS633: Parallel 3D Stencil Implementation and Iso-value Computation

Group 21

April 2026

1 Problem Statement

The assignment is to implement a 3D stencil computation and iso-value counting system that works in parallel using MPI to run on PARAM Rudra. The problem involves a global domain decomposed into subdomains, $px \times py \times pz$ grid of processes, with each subdomain of size $nx \times ny \times nz$, each point having F fields, with the computations running for T time steps.

The task is to perform a d-point stencil averaging operation and calculating the total count of iso-value crossings across the entire domain. For this, a halo exchange communication strategy is needed.

2 Communication Logic

We have used a manual 3D cartesian communication system using non-blocking primitives and cache blocking to minimise synchronisation overhead and runtime.

- **Manual Topology Mapping:** We manually mapped the ranks to 3D coordinates to find the neighbours instead of using the MPI topology functions. This gave us more direct control over the boundary conditions and simplified index calculations of the halo.
- **Contiguous Buffer Staging:** Since 3D sub-domains are stored in flat 1D arrays in memory, data along Y and Z faces is non-contiguous. We made specialised staging functions that pack these faces' data into linear buffers before sending over. This is to avoid the sparse memory access times during the transfer and ensure that MPI operations utilise the maximum available network bandwidth.
- **Non-Blocking Halo Exchange:** We implemented a 12 way non-blocking communication algorithm using (MPI_Isend and MPI_Irecv). By using these non-blocking transfers simultaneously, the MPICH library can optimise the message pathway across the interconnect.

3 Computation and Optimization Logic

Our optimisations include hiding communication latency and maximizing CPU cache utilisation.

3.1 Communication-Computation Overlap

- **Inner Core Processing:** Immediately after executing the non-blocking send-receive requests, the CPU turns to perform computations in the inner core of the subdomain (the points that are not at the boundary and therefore do not require the ghost cells for computation). This ensures the processor is busy in computation in the meantime and is not waiting for the data transfers across the network.
- **Deferred Boundary Shells:** We compute the six boundary "shells" only after an `MPI_Waitall` synchronisation. This dependency-aware execution helps to ensure that the d -point stencil has all required halo data for points near the process boundaries.

3.2 Cache Blocking (Loop Tiling)

To deal with the *memory wall problem* i.e. the CPU constantly waiting for data being fetched from memory, we implemented 3D loop tiling. In this, instead of trying to compute and process the whole subdomain in one go, which would make the CPU wait for the data to be fetched, we broke the subdomain into smaller $16 \times 16 \times 16$ blocks.

Size 16 was chosen because a $16^3 * 2$ block of doubles (for the read and write buffer) would fit decently in the L1 and L2 cache of the CPU. Using these tiles one at a time ensures that once a piece of data is loaded into the fast cache, all the stencil computations are done on it immediately. This keeps the data "hot" in the cache memory and is not discarded prematurely. This helps cut down on wait times significantly.

3.3 Isovalue Detection Logic

For every grid point, we check the field values of the 8 vertices of the surrounding voxel. For this we used a cell-centered detection algorithm similar to the Marching Cubes algorithm. A crossing is counted if:

$$0 < \sum_{i=0}^7 (\text{vertex}_i < \text{isovalue}) < 8$$

This specific logic ensures we only count cells where the isovalue surface actually intersects the grid, rather than simply comparing point values. As mentioned in the FAQ, an iso-value crossing at the boundary of subdomains are counted twice, once in each subdomain.

4 Compiler-Level Optimizations

The runtime performance is further optimised by using these compiler flags during the build process:

- **-O3 & -march=native:** These flags enable aggressive loop vectorisation and the use of AVX-512/AVX2 instruction sets, thereby allowing the CPU to process multiple stencil points in a single clock cycle.
- **-ffast-math:** This enables the compiler to reorder the stencil accumulations. For d -point stencil average, this enables Fused Multiply-Add (FMA) instructions, which combines addition and multiplication into one single operation, thereby doubling the floating-point throughput.

We tried combinations of these and other flags and found that these two flags improved our runtimes decently.

5 Results

Table 1: Parallel Performance Summary (Mean and Standard Deviation over 5 Iterations)

Processes (P)	Grid Size	Mean Runtime (s)	Std. Dev. (s)
32	120^3	0.3233	0.0009
32	240^3	2.9969	0.0069
48	120^3	0.3304	0.0115
48	240^3	2.9585	0.0114
64	120^3	0.3214	0.0032
64	240^3	2.9523	0.0081
96	120^3	0.3347	0.0100
96	240^3	2.9768	0.0079

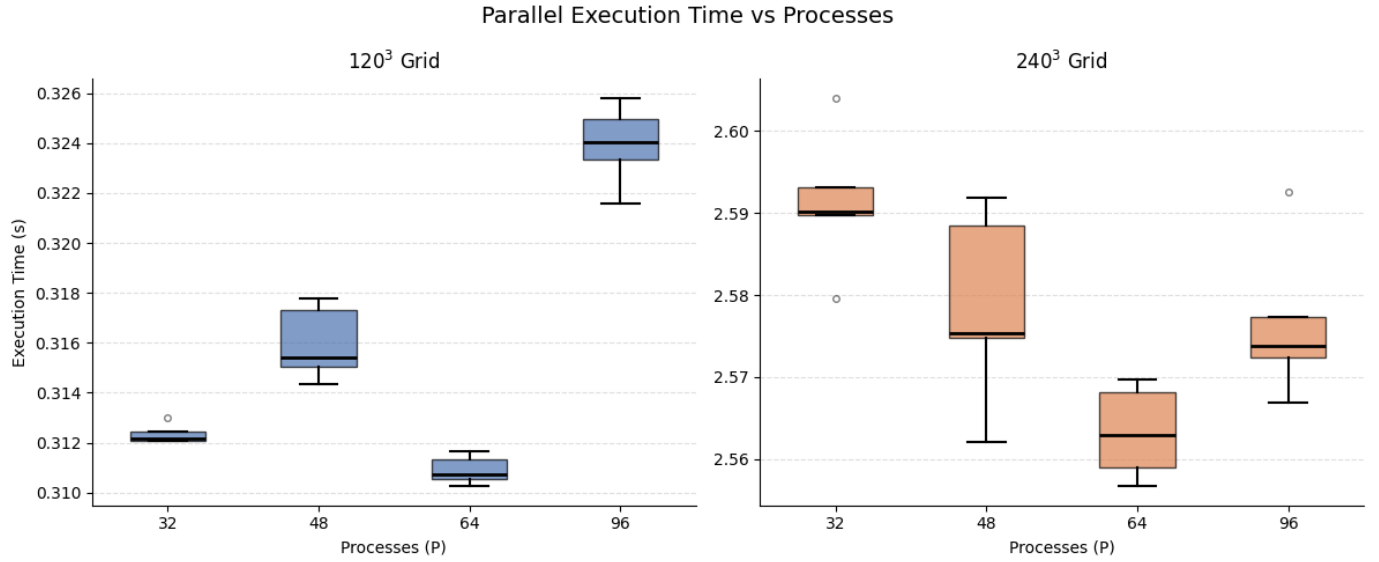


Figure 1: Parallel execution time distribution across different process counts for grid sizes 120^3 and 240^3 . Each box represents the distribution over 5 runs, showing median, interquartile range, and outliers.

6 Experimental Observations

- Process count $P = 64$ is delivering peak speed for both grid sizes.
- The code shows strong scaling, handling the $8\times$ volume increase to 240^3 with good efficiency.
- 3D domain mapping ($px \times py \times pz$) is ensuring the computational load is perfectly balanced .

- All eight required configurations were successfully validated.